

# Resumo do Capítulo 1: Introdução ao Big Data

## Introdução

O Capítulo 1 introduz os princípios e a importância do Big Data, explicando que a sua resolução requer um novo paradigma de computação e o princípio da **Localidade dos Dados**.

## O Que É Big Data?

Não existe uma definição padrão para “Big Data”. No entanto, o termo envolve dados cujo volume, diversidade e complexidade exigem novas técnicas, algoritmos e análises para extrair conhecimento valioso (oculto).

## A Definição dos V’s

O Big Data é frequentemente definido pelos **3 V’s**, estendidos para **5 V’s**:

1. **Volume (Data at rest)**: Refere-se às vastas quantidades de dados gerados. Os conjuntos de dados são grandes demais para o armazenamento tradicional, exigindo sistemas distribuídos.
2. **Velocity (Data in motion)**: A velocidade com que os dados são gerados e precisam de ser analisados. Decisões tardias implicam oportunidades perdidas, especialmente com *data streams* contínuos (e.g., sensores).
3. **Variety (Data in many forms)**: Os dados podem ter vários formatos e estruturas, incluindo dados Estruturados (tabelas), Semi-Estruturados (ficheiros XML) e Não-Estruturados (texto, áudio, vídeo).
4. **Veracity (Data in doubt)**: A incerteza sobre a qualidade dos dados, que podem estar em falta, ser ambíguos ou incorretos (e.g., erros tipográficos ou sarcasmo em redes sociais).
5. **Value (Data in use)**: A motivação mais importante. O Big Data deve resultar em melhores estatísticas/modelos, novos insights e novas oportunidades para a investigação e indústria.

## Exemplo — Google Flu

Em 2009, o Google utilizou a correlação de termos de pesquisa para detetar a gripe H1N1 em tempo real, ilustrando a necessidade de paralelizar a análise de dados (usando Hadoop MapReduce).

## Como Lidar com Big Data: Escalabilidade

O problema central é a **escalabilidade** para grandes conjuntos de dados.

- **Solução:** O princípio do *Dividir para Conquistar* (*Divide-and-Conquer*) é necessário, pois uma única máquina não consegue gerir grandes volumes de dados de forma eficiente.

### Modelos de Escalabilidade

- **Scale-up (Escalabilidade Vertical):** Utiliza ferramentas tradicionais numa única máquina mais potente. É mais caro, não é tolerante a falhas (*fault-tolerant*) e tem limites de atualização.
- **Scale-out (Escalabilidade Horizontal):** Usa múltiplas máquinas (nós) mais baratas ligadas por uma rede (*computação distribuída*). Esta é a base das Tecnologias Big Data. É mais barato, fácil de expandir e permite a tolerância a falhas.

## O Paradigma de Computação do Big Data

O Big Data difere da **Computação de Alto Desempenho (HPC)** nas suas características fundamentais.

| Característica | Big Data                                                                     | HPC (High Performance Computing)                 |
|----------------|------------------------------------------------------------------------------|--------------------------------------------------|
| Foco           | Trabalhos intensivos em dados                                                | Trabalhos intensivos em computação (simulação)   |
| Falhas de H/W  | Comuns                                                                       | Surpresa                                         |
| Hardware       | <i>Commodity hardware</i> (comercial)                                        | Hardware especializado                           |
| Movimento      | O trabalho é movido para onde os dados estão ( <b>Localidade dos Dados</b> ) | Os dados são movidos para onde serão processados |

### Princípio da Localidade dos Dados

O princípio da **Localidade dos Dados** (*Data Locality*) é crucial, pois é preciso evitar ao máximo as transferências de dados entre máquinas.

### Novo Modelo de Programação: MapReduce

O novo modelo de programação necessário é o **MapReduce**. A ideia principal é que:

“Mover a computação é mais barato do que mover a computação e os dados ao mesmo tempo.”

Os dados são distribuídos, as funções são distribuídas, e cada nó de computação trabalha apenas com os dados armazenados localmente.

## Resumo do Capítulo 2: MapReduce

O **MapReduce** é um modelo de programação paralela e um paradigma de computação funcional (e.g., Haskell) projetado para computação distribuída com Big Data. O principal objetivo é aplicar operações em vastos conjuntos de dados.

### Princípios Fundamentais do MapReduce

#### 1. Estratégia Divide & Conquer (Dividir e Conquistar)

O MapReduce utiliza esta estratégia para processamento paralelo:

- **Dividir (Divide):** Particionar o conjunto de dados em blocos menores e independentes para serem processados em paralelo (**Fase Map**).
- **Conquistar (Conquer):** Combinar, fundir ou agregar os resultados da fase anterior (**Fase Reduce**).

#### 2. Localidade dos Dados (Data Locality)

MapReduce baseia-se fortemente neste princípio:

- A regra fundamental é que *“Mover a computação é mais barato do que mover a computação e os dados ao mesmo tempo.”*
- Os dados são distribuídos entre os nós (*nodes*) do sistema de ficheiros distribuído, e as funções de processamento são distribuídas para esses nós.
- Cada nó de computação trabalha apenas com os dados armazenados localmente. Apenas os dados estritamente necessários são movidos através da rede.

#### 3. Transparência e Tolerância a Falhas (Fault-Tolerance)

É desejável que o sistema seja transparente para os programadores e tolerante a falhas.

- Em sistemas de larga escala (e.g., 10.000 nós), falhas de hardware (ou nós lentos) são comuns e devem ser geridas automaticamente.

### O Funcionamento MapReduce: Pares Chave-Valor

O MapReduce opera sobre pares  $\langle \text{key}, \text{value} \rangle$  (chave-valor). O utilizador precisa de especificar duas funções principais: **map** e **reduce**.

### A. A Função Map (Fase de Mapeamento)

- **Entrada/Saída:** `map: (k, v) → list[k', v']`.
- A função `map` pega um par de entrada  $(k, v)$  e aplica uma operação, retornando uma lista de pares chave-valor intermediários  $(k', v')$ .
- A chave de entrada `k` para a função `map` é tipicamente metadados (e.g., número da linha) e geralmente não é de grande interesse.
- Dependendo do tamanho dos dados de entrada, haverá múltiplas tarefas **MAP**.

### B. A Fase Shuffle (Agrupamento Automático)

- Esta fase é automática e ocorre entre o **Map** e o **Reduce**.
- É responsável por agrupar os pares intermediários pela chave  $(k')$ .
- Transforma `list[k', v']` em  $(k', list[v'])$ .
- Todos os pares com a mesma chave  $(k')$  serão enviados para o mesmo nó **reducer**.
- O *shuffle* pode ser um gargalo (*bottleneck*) se for necessário enviar grandes quantidades de dados pela rede, pois é onde ocorre toda a transferência de dados.

### C. A Função Reduce (Fase de Redução)

- **Entrada/Saída:** `reduce: (k', list[v']) → list[k', v'']`.
- A função `reduce` recebe o par  $(k', list[v'])$  e reduz a lista de valores  $v'$  para um único  $v''$ , devolvendo um novo par  $(k', v'')$ .
- É aplicada independentemente a cada chave diferente.
- A função `reduce` deve ser **comutativa** e **associativa**.
- O número de tarefas **REDUCE** será definido pelo número de chaves distintas  $(k', value)$ .

### Otimização: Combiners

- **Combiners** são agregadores locais (*local aggregators*).
- Funcionam exatamente como a função `reduce`, mas são aplicados localmente após a fase `map`.
- O objetivo principal é reduzir a quantidade de dados enviados através da rede, atacando o gargalo do *shuffle*.

- São particularmente úteis quando há repetição significativa nas chaves intermediárias (e.g., no exemplo do **Word Count**).
- O seu uso é **opcional**.

## O Exemplo “Hello World” do Big Data: Word Count

O cálculo da frequência de palavras num documento (**Word Count**) é o exemplo clássico.

- **Objetivo:** Contar a frequência de cada palavra (e.g., {‘Welcome’: 2, ‘Big’: 4, ‘World’: 3}).
- Se o documento for muito grande, o MapReduce é usado para paralelizar o processo, dividindo os dados em *chunks* e processando-os em máquinas diferentes.
- A solução MapReduce requer que o trabalho seja dividido (*split*) e os resultados sejam fundidos (*merge*).

### Mensagens-Chave para a Frequência

O MapReduce é um modelo de programação que oferece um novo nível de abstração para o programador. As suas características distintivas são:

- Localidade dos Dados (transparente) e Redundância (HDFS);
- Tolerância a Falhas;
- Paralelização Automática;
- Escalabilidade em *data centers* ou *clusters*;
- O fluxo de trabalho principal é: **Map**, **Shuffle** e **Reduce**. É crucial entender a operação **shuffle** para otimizar a transferência de dados.

## Ficha 2: MapReduce

### 1. Sequência correta de estágios

A sequência correta de estágios em um programa MapReduce é: **d. map, shuffle, reduce.**

### 2. Verdadeiro/Falso sobre MapReduce

1. MapReduce garante que não haverá movimento de dados pela rede. **Falso.** A fase shuffle pode envolver movimento de dados pela rede. O princípio da localidade de dados visa minimizar as transferências, mas não as elimina.
2. É baseado em programação orientada a objetos. **Falso.** MapReduce é baseado em programação funcional.
3. Ao contrário do MPI, MapReduce requer programação explícita da comunicação entre nós. **Falso.** MapReduce é transparente para o programador, não exigindo programação explícita de comunicação. O MPI (Message Passing Interface) é que requer que o programador lide com a comunicação.
4. Hadoop implementa o modelo de programação MapReduce. **Verdadeiro.**

### 3. Pares Chave-Valor (V/F)

1. As chaves de entrada para a fase map são extremamente importantes na minimização do movimento de dados. **Falso.** A chave de entrada ( $k$ ) geralmente não é de interesse (pode ser apenas metadados, como um número de linha ou nome de arquivo).
2. As chaves em todos os estágios (map, shuffle e reduce) precisam ser numéricas. **Falso.** As chaves e valores podem ser de qualquer tipo de dado.
3. Os valores de saída da fase map podem ser uma tupla contendo qualquer tipo de dado. **Verdadeiro.**
4. A fase reduce não funciona em pares chave-valor. **Falso.** A fase reduce trabalha com pares  $(k', list[v'])$  e retorna novos pares  $(k', v'')$ .

### 4. Fase Map (V/F)

1. Você pode acessar qualquer outro par chave-valor residente em qualquer outra partição. **Falso.**
2. Pode gerar um ou mais pares chave-valor a partir de um único input  $\langle k, v \rangle$ . **Verdadeiro.**

3. Os valores de saída desta fase são geralmente irrelevantes e desconsiderados. **Falso.**
4. No Hadoop, um processo chamado mapper executa esta função em uma partição dos dados. **Verdadeiro.**

## 5. Fase Shuffle (V/F)

1. Os programadores têm que especificar o comportamento desta fase como uma função. **Falso.**
2. Pode envolver movimento de dados pela rede. **Verdadeiro.**
3. Agrupa todos os pares com os mesmos valores e cria uma lista com as suas chaves. **Falso.** Agrupa pelos mesmos *valores* de chave e cria uma lista de *valores*.
4. O seu desempenho pode ser influenciado pela forma como definimos a função map. **Verdadeiro.**

## 6. Concatenar processos MapReduce

Problemas complexos frequentemente requerem múltiplas etapas, embora o modelo não explique explicitamente como concatenar processos.

## 7. Transparência e Tolerância a Falhas (MapReduce vs. MPI)

**Transparência:** MapReduce fornece abstração e automatiza a comunicação e paralelização. O MPI requer comunicação explícita (e.g., *Scatter/Gather*).

**Tolerância a Falhas:** MapReduce é tolerante a falhas e reexecuta tarefas em caso de falha de nós. O MPI não possui essa tolerância.

## 8. O papel dos Combiners

Combiners são agregadores locais que funcionam como a função reduce. Reduzem localmente os dados intermediários, minimizando o tráfego da fase shuffle. O seu uso é opcional.

## 9. Por que a função reduce deve ser comutativa e associativa

A função reduce deve ser comutativa e associativa para garantir resultados consistentes independentemente da ordem de processamento. Exemplo de função problemática: subtração ( $\lambda x, y : x - y$ ).



## Resolução de Exercícios Práticos

### 10. Output da Função Map

Função:  $map(k, v) = (v * k, v)$  Input:  $[(0, 15), (15, 0), (2, 0)]$

- $(0, 15) \rightarrow (0, 15)$
- $(15, 0) \rightarrow (0, 0)$
- $(2, 0) \rightarrow (0, 0)$

**Output do Map:**  $[(0, 15), (0, 0), (0, 0)]$

### 11. Output da Fase Shuffle

Agrupar os pares pela chave: **Output do Shuffle:**  $(0, [15, 0, 0])$

### 12. Output da Fase Reduce (Multiplicação)

Função:  $\lambda x, y : x * y$  Input:  $(0, [15, 0, 0])$  Redução:  $15 * 0 * 0 = 0$  **Output:**  $(0, 0)$

### 13. Output da Fase Reduce (Subtração)

Função:  $\lambda x, y : x - y$  Input:  $(0, [15, 0, 0])$  Redução:  $(15 - 0) - 0 = 15$  **Output:**  $(0, 15)$

**Problemas:** A subtração não é comutativa nem associativa.

### 14. Função special(x, y): Associativa e Comutativa?

Função: Retorna o maior valor ( $\max(x, y)$ ). **Sim.** É associativa e comutativa.

### 15. MapReduce (Cálculo do Comprimento Total)

Input:

```
(1, "Welcome to the Big")
(2, "World of Big Big")
(3, "Data Welcome World Bye")
```

Map Output:

- $(1, 7), (1, 2), (1, 3), (1, 3)$
- $(1, 5), (1, 2), (1, 3), (1, 3)$
- $(1, 4), (1, 7), (1, 5), (1, 3)$

Shuffle Output:  $(1, [7, 2, 3, 3, 5, 2, 3, 3, 4, 7, 5, 3])$

Reduce:  $\lambda x, y : x + y$  Soma:  $7 + 2 + 3 + 3 + 5 + 2 + 3 + 3 + 4 + 7 + 5 + 3 = 47$  **Output**

**Final:**  $(1, 47)$

**Tarefa Desempenhada:** Soma do comprimento de todas as palavras nas frases de entrada.

# Resumo do Capítulo 3: Hadoop

## O que é Hadoop

- Hadoop é um *framework* de código aberto escrito em Java, concebido para o armazenamento e processamento distribuído de conjuntos de dados muito grandes (*very large data sets*).
- Hadoop implementa o modelo de programação **MapReduce**.

## Componentes Principais do Hadoop

O *framework* Hadoop é composto por vários módulos:

- **Hadoop Common**: Utilitários e bibliotecas de suporte.
- **Hadoop Distributed File System (HDFS)**: O sistema de ficheiros distribuído.
- **Hadoop YARN (Yet Another Resource Negotiator)**: O gestor de recursos (introduzido na Versão 2).
- **Hadoop MapReduce**: A implementação do modelo de programação.

## O Paradigma MapReduce

- MapReduce é um modelo de programação para *big data* que utiliza duas funções principais: **map** e **reduce**.
- É apropriado para o processamento de grandes volumes de dados devido à *localidade dos dados* (*data locality*), transparência (esconde a complexidade) e tolerância a falhas.

## YARN (Gestão de Recursos)

YARN foi introduzido na versão 2 para retirar a administração de recursos do módulo MapReduce, permitindo que outros projetos utilizem YARN e HDFS. A sua organização baseia-se em quatro conceitos-chave:

1. **Resource Manager (RM)**: Controla os recursos disponíveis em todo o *cluster*.
2. **Node Manager (NM)**: Lança e monitoriza os processos atribuídos aos *workers* (um por nó).
3. **Container**: É um subconjunto de recursos do *cluster*.

4. **Application Master (AM):** Gere uma aplicação específica (correndo num *container*) e é responsável pela tolerância a falhas.

O processo de execução MapReduce (na V2) envolve o RM a alocar um *container* para o AM, que depois solicita *containers* para executar as tarefas em diferentes nós.

## HDFS (Hadoop Distributed File System)

O HDFS é um sistema de ficheiros de rede distribuído, construído a partir de *hardware commodity* (baixo custo), onde a falha de nós é a norma.

### Princípios Chave de Design

- Ficheiros muito grandes (tipicamente vários GB).
- Padrão de acesso **escrever-uma-vez, ler-muitas-vezes** (*write-once, read-many-times*).
- Alta taxa de transferência (*High throughput*) é mais importante do que baixa latência.
- Implementa o princípio da *localidade dos dados* (*data locality*).

### Tolerância a Falhas e Blocos

- A prevenção de perda de dados é conseguida através de replicação (padrão 3x, incorrendo numa sobrecarga de 200% no armazenamento).
- Mais recentemente, utiliza-se a **codificação de apagamento** (*erasure coding*) para reduzir o armazenamento necessário (aprox. 1.5x).
- O bloco é a unidade mínima de dados no HDFS. Os blocos são grandes (64MB ou 128MB por padrão) para minimizar o tempo de procura (*seek time*).

### Arquitetura Master-Worker

- **Name Node (Master):** Armazena o *namespace* do sistema de ficheiros e a localização dos blocos de dados.
- **Data Nodes (Workers):** Armazenam e recuperam os blocos de dados.

## Diferenças entre HDFS V1 e V2

- Na V1, o Name Node era o ponto único de falha.
- A partir da V2, foram adicionados:
  - **HDFS Federation**: múltiplos *Name Spaces* independentes.
  - **HDFS High-Availability**: sincronização contínua entre Name Nodes, permitindo a recuperação em segundos.

## Limitações de Hadoop MapReduce

Embora o MapReduce ofereça simplicidade e tolerância a falhas, não resolve todos os tipos de problemas. As suas limitações incluem:

- Estilo de programação pesado (*cumbersome programming style*).
- Ineficiência de velocidade para aplicações que partilham dados em múltiplos passos, como algoritmos iterativos, modelos de grafos ou consultas interativas.
- É lento devido à replicação, serialização e I/O de disco.

Estas limitações levaram ao desenvolvimento de novas plataformas, como o **Spark**, que superam estes desafios. O Hadoop, contudo, é a base do ecossistema de ferramentas *Big Data*.

## Ficha 3: Hadoop

### 1. Componentes/Módulos Principais do Apache Hadoop

Os módulos principais do Apache Hadoop são:

- Hadoop Common.
- Hadoop Distributed File System (HDFS).
- Hadoop YARN – o gerenciador de recursos (a partir da V2).
- Hadoop MapReduce – o modelo de programação.

### 2. Verdadeiro/Falso sobre Hadoop

- a) **Hadoop segue uma estratégia scale-up... Falso.** Hadoop utiliza *clusters shared-nothing* construídos a partir de hardware de baixo custo, o que é característico da estratégia *scale-out*.
- b) **É necessário um cluster para testar Hadoop. Falso.** Hadoop pode ser executado em modo Local / Standalone, em uma única JVM, o que é útil para testes e depuração.
- c) **Não se pode executar mais de um job MapReduce ao mesmo tempo. Falso.** O YARN gerencia os recursos para todas as aplicações em execução, permitindo a concorrência de múltiplos jobs.
- d) **Hadoop usa o conceito de containers para isolamento de recursos. Verdadeiro.** O *container* é um conceito chave no YARN, representando um subconjunto de recursos do cluster.

### 3. Tipos de Dados Próprios do Hadoop

O MapReduce do Hadoop usa tipos de dados próprios (como `IntWritable`) em vez de tipos Java padrão (como `Integer`), pois é necessário serializar objetos para a comunicação e movimento de dados entre os nós do cluster. (Os tipos `Writable` são otimizados para esta serialização e comunicação de rede.)

### 4. Necessidade de Serialização

Precisamos serializar objetos em um programa Hadoop MapReduce para permitir a comunicação e o movimento de dados (pares chave-valor) entre os diferentes nós da rede, especialmente durante a fase de *shuffle*.

## 5. Verdadeiro/Falso sobre YARN

- a) **Apenas jobs MapReduce podem ser executados no YARN. Falso.** A introdução do YARN na V2 permitiu que outros projetos usassem o gerenciamento de recursos e o HDFS sem usar MapReduce.
- b) **O Node Manager reside no nó mestre. Falso.** O Node Manager reside nos nós *worker* e é responsável por lançar e rastrear processos atribuídos a esses nós.
- c) **Os containers de uma dada aplicação devem sempre estar localizados no mesmo nó worker. Falso.** O Application Master requisita *containers* para executar as tarefas da aplicação, que podem ser localizados em nós diferentes.
- d) **O Application Master é o mesmo para todos os jobs no cluster. Falso.** O Application Master gerencia uma aplicação particular.

## 6. Necessidade de Containers no YARN

Os *containers* são necessários porque representam um subconjunto de recursos do cluster. Eles são a unidade onde o YARN aloca e executa as tarefas de uma aplicação, fornecendo isolamento de recursos.

## 7. Por que o Application Master não roda no nó mestre

O Resource Manager aloca um único *container* em um nó *worker* (não no nó mestre) onde o Application Master é executado. Isso é feito para que o nó mestre (onde residem o Resource Manager e o Name Node) não tenha seus recursos sobrecarregados pelos processos de aplicação.

## 8. Principais Etapas da Execução de um Processo MapReduce no YARN

1. O cliente lança o processo e se conecta com o Resource Manager.
2. O Resource Manager aloca um único *container* em um nó *worker* onde o Application Master é executado.
3. O Application Master requisita os *containers* necessários para executar todas as tarefas (em nós diferentes).
4. Todas as tarefas são executadas nos *containers*. Os *containers* são liberados após a conclusão das tarefas.

5. O Application Master finaliza e libera o seu *container* quando todas as tarefas são completadas.

## 9. Modos de Execução do Hadoop

Os três modos de execução disponíveis no Hadoop são:

1. **Local / Standalone** – executado em uma única JVM.
2. **Pseudo-distributed** – simulador de *cluster*.
3. **Distributed** – modo de execução real em *cluster*.

O modo usado para depuração do código é o **modo Local / Standalone**.

## 10. Verdadeiro/Falso sobre HDFS

- a) **O HDFS é bom para ler arquivos grandes, mas não para mudanças aleatórias. Verdadeiro.** O HDFS foi projetado para *write-once, read-many-times*, priorizando alto *throughput* de dados. Não é adequado para modificações arbitrárias ou múltiplas escritas concorrentes.
- b) **O HDFS foi projetado para ter uma latência muito baixa. Falso.** O HDFS prioriza o alto *throughput* (para dados em massa) em detrimento da baixa latência.
- c) **O conceito de bloco é importante no HDFS para fornecer tolerância a falhas. Verdadeiro.** A tolerância a falhas é alcançada pela replicação (tipicamente 3x) de cada bloco em máquinas diferentes.
- d) **O Name Node sabe o paradeiro de todos os arquivos no HDFS. Verdadeiro.** O Name Node armazena o *Filesystem namespace* e as localizações dos blocos de dados.

## 11. Armazenamento de um Arquivo de 5TB (5 Nós x 1TB)

Não. O cluster possui 5 TB de capacidade bruta. O HDFS usa replicação de dados para tolerância a falhas (fator padrão 3x). Para armazenar um arquivo de 5 TB com replicação 3x, seriam necessários 15 TB de espaço físico, o que excede a capacidade total do cluster.

## 12. Distribuição e Espaço Ocupado (4 Nós, R=3)

Assumindo o tamanho de bloco padrão de 128 MB:

- Ficheiro 1 (200 MB): Requer  $\lceil 200/128 \rceil = 2$  blocos.



- Ficheiro 2 (390 MB): Requer  $\lceil 390/128 \rceil = 4$  blocos.
- Total de Blocos:  $2 + 4 = 6$  blocos.
- Espaço Ocupado (Replicação 3x):  $(200 + 390) \times 3 = 590 \times 3 = 1770$  MB (1.77 GB).

A distribuição seria a dispersão desses 6 blocos pelos 4 *Data Nodes*, garantindo que 3 cópias de cada bloco existam em nós diferentes.

### 13. Motivo do Grande Tamanho de Bloco (128MB)

O HDFS utiliza um grande tamanho de bloco (tipicamente 128 MB) para minimizar o tempo de busca (*seek time*). Ao ler um bloco grande, o tempo gasto buscando o bloco é insignificante comparado ao tempo gasto na transferência dos dados, aumentando assim o *throughput* de leitura.

### 14. Tolerância a Falhas no Hadoop

A tolerância a falhas é alcançada em dois níveis:

1. **Nível de Dados (HDFS):** Utiliza replicação (o padrão é 3x) de blocos de dados em diferentes nós. Versões mais recentes também usam *Erasure Coding* para reduzir o overhead de armazenamento.
2. **Nível de Processamento (YARN/MapReduce):** O Application Master é responsável por garantir a tolerância a falhas. Se um nó falhar, as tarefas podem ser reexecutadas em outros nós do cluster.

### 15. Duas Razões para Não Usar Hadoop MapReduce

Pode-se não querer usar MapReduce se a solução exigir:

1. **Velocidade e Eficiência em Múltiplas Etapas:** MapReduce é ineficiente/lento para aplicações que compartilham dados em múltiplas etapas, como algoritmos iterativos, modelos gráficos ou consultas interativas.
2. **Baixa Latência:** É lento devido à replicação, serialização e I/O de disco.
3. **Programação Simplificada:** O estilo de programação é considerado complicado (*cumbersome*).

# Resumo do Capítulo 4: Spark

## Introdução e Motivação do Spark

O **Apache Spark** surgiu para superar as limitações do modelo **MapReduce (Hadoop)** em certos cenários.

- **Problema do MapReduce:** A execução de tarefas iterativas, consultas interativas ou streaming no MapReduce requer múltiplos acessos ao disco (*disk accesses*), o que torna a E/S (*I/O*) lenta.
- **Solução Spark:** O Spark permite manter os dados na memória principal (*main memory*), tornando-o muito mais rápido do que serializar cada fase numa unidade de disco.

## O Que É Apache Spark?

- O Spark é um **Motor de Computação em Cluster Rápido e Expressivo** (*Fast and Expressive Cluster Computing Engine*).
- É um **motor de processamento de dados** (*Data Processing Engine*) para Big Data.
- É compatível com o Apache Hadoop e estende o paradigma MapReduce para processamento interativo, iterativo e *stream processing*.
- **Armazenamento e Gestão:** O Spark não possui um sistema de ficheiros distribuído próprio, utilizando sistemas existentes como HDFS ou NoSQL. Também não exige o Hadoop, sendo compatível com vários gestores de cluster, como YARN (Hadoop), Mesos ou o modo Standalone.
- **APIs:** Oferece APIs ricas em Scala, Java, Python (PySpark) e R.

## Arquitetura de Aplicação Spark

Uma aplicação Spark é composta por dois programas:

1. **Driver:** Executa o código sequencial e funciona como o ponto de entrada principal.
  2. **Workers (Executores):** Realizam as operações paralelas.
- **SparkContext (sc):** É o principal ponto de entrada para a funcionalidade Spark. É criado no Driver e estabelece a conexão com o gestor de cluster, solicitando Executores.

- **Executores (Executors):** São processos que realizam a computação e armazenam os dados nos nós (*workers*). Cada programa tem os seus próprios executores, garantindo isolamento.

## RDDs (Resilient Distributed Datasets)

Os RDDs são a principal abstração de dados do Spark, fornecendo memória distribuída.

- **Definição:** Um RDD é uma coleção de elementos tolerante a falhas que pode ser operada em paralelo e armazenada em cache para reutilização futura.
- **Características Chave:**
  - **Imutável:** Depois de criado, não pode ser alterado.
  - **Tolerância a Falhas:** Os RDDs são reconstruídos automaticamente em caso de falha. Isto é garantido pela manutenção da *Linhagem (Lineage)*, que regista todas as transformações usadas.
  - **Distribuição:** Os RDDs são distribuídos por todo o cluster. A distribuição é feita em **Partições**.
- **Criação de RDDs:** Podem ser criados através da paralelação de coleções no programa Driver, referenciando um conjunto de dados num sistema de armazenamento externo (HDFS, HBase) ou transformando um RDD existente.

## Operações com RDDs

As operações com RDDs dividem-se em dois tipos:

| Tipo de Operação | Característica Principal / Exemplos                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transformações   | Operações Lazy (Preguiçosas). Criam um novo RDD a partir de um existente. A computação só é executada quando uma Ação é chamada. Exemplos: <code>map(func)</code> , <code>filter(func)</code> , <code>flatMap(func)</code> , <code>union()</code> , <code>distinct()</code> . Disparam a computação e devolvem um resultado ao nó Driver ou escrevem-no no armazenamento. Exemplos: <code>count()</code> , <code>collect()</code> , <code>reduce(func)</code> , <code>first()</code> , <code>take(n)</code> , <code>saveAsTextFile()</code> . |
| Ações            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

### Diferenciação Importante:

- `map()` retorna um novo dataset onde cada item de entrada é mapeado para um item de saída.

- `flatMap()` é semelhante, mas cada item de entrada pode ser mapeado para zero ou mais itens de saída (retorna uma sequência em vez de um único item).

## RDDs de Chave-Valor (Key-Value)

O Spark suporta pares Chave-Valor, onde cada elemento de um RDD é uma tupla ((key, value)).

### Transformações de Chave-Valor (geralmente requerem Shuffle)

Estas operações tendem a ser de dependência *wide* (larga):

- `groupByKey()`: Retorna um novo RDD de tuplas (k, iterable(v)). Requer um *shuffle*.
- `reduceByKey(func)`: Agrega os valores de cada chave k usando uma função func.
- `sortByKey([ascending])`: Ordena o RDD pela chave.
- **Joins (Uniões):**
  - `join(rdd)`: União interna (Inner join), a chave deve estar em ambos os RDDs.
  - `leftOuterJoin(rdd)`, `rightOuterJoin(rdd)`, `fullOuterJoin(rdd)`: Uniões externas.

### Ações de Chave-Valor

- `countByKey()`: Conta o número de elementos para cada chave, retornando um dicionário.
- `collectAsMap()`: Recolhe o RDD como um dicionário, mas fornece apenas um dos valores por chave.
- `lookup(key)`: Retorna o valor (ou valores) associado a uma determinada chave.

## Conceitos Avançados

### A Importância do Caching

- O caching não acontece por padrão. Se um RDD for usado e computado várias vezes (por exemplo, dentro de um laço iterativo), o Spark irá recomputá-lo.
- `rdd.cache()`: Deve ser usado em RDDs que são reutilizados para armazenar o resultado em memória ou disco, aumentando a eficiência.

## Variáveis Partilhadas (Shared Variables)

Alterações feitas por executores em variáveis globais não são visíveis no Driver. O Spark fornece duas variáveis especiais para partilha eficiente:

1. **Variáveis Broadcast:** Variáveis só de leitura enviadas eficientemente para todos os executores e armazenadas em cache localmente. São ideais para tabelas de *look-up* grandes.
2. **Acumuladores (Accumulators):** Permitem agregar valores dos executores no Driver (tipicamente para contadores ou somas paralelas). São apenas de escrita para as tarefas.

## Partições

- As partições determinam o grau máximo de paralelismo que pode ser alcançado.
- `mapPartitions(func)`: Aplica uma função a todos os elementos de uma partição de uma só vez. Pode ser mais eficiente em certos casos, pois reduz a sobrecarga de E/S por item.

## Funcionamento Interno (Internal Working)

O Spark otimiza a execução criando um **Grafo Acíclico Dirigido (DAG)** para cada aplicação.

- **Etapas (Stages):** O DAG é dividido em etapas.
- **Dependências:** As transformações são classificadas em:
  1. **Dependências Narrow (Estreitas):** Cada partição de entrada contribui para apenas uma partição de saída. Não há troca de dados (*shuffle*) no cluster.
  2. **Dependências Wide (Largas):** As partições de entrada contribuem para múltiplas partições de saída, exigindo um *shuffle*. Transformações como `sort`, `reduceByKey`, `groupByKey` e `join` são de dependência wide e definem os limites de uma etapa Spark.

## Ficha 4: Spark

### 1. Verdadeiro/Falso sobre Spark

1. Spark não é apenas um motor de processamento de dados, pois vem com seu próprio sistema de arquivos distribuídos. **Falso.** Spark é apenas um motor de processamento de dados. Ele não possui um sistema de arquivos distribuído próprio; em vez disso, usa sistemas existentes como HDFS ou NoSQL.
2. Spark estendeu o modelo de programação MapReduce com operações adicionais. **Verdadeiro.** Spark estende o paradigma MapReduce para processamento interativo, iterativo e de streaming, oferecendo operações mais ricas (e.g., `join`, `sample`, etc.).
3. Spark visa mitigar os problemas de serialização de dados para o disco que o Hadoop MapReduce sofre. **Verdadeiro.** Spark armazena dados na memória principal (*in-memory storage*), muito mais rápido do que serializar cada estágio para o disco.
4. Spark precisa do YARN para executar suas aplicações em um cluster. **Falso.** Spark é compatível com YARN, mas também pode funcionar com Mesos ou em modo Standalone.

### 2. Spark vs. Hadoop MapReduce

- Spark é preferido porque oferece armazenamento em memória, tornando-o mais eficiente e rápido do que Hadoop MapReduce, que serializa dados para o disco em cada estágio.
- Mesmo que a aplicação MapReduce tenha apenas um estágio, espera-se que o Spark ainda seja mais rápido.
- Hadoop envolve muitas operações de I/O para disco, mesmo entre as fases `map` e `reduce`. Spark mitiga isso usando memória.

### 3. Uso do HDFS com Spark

- Spark não exige o uso de HDFS.
- A maioria dos clusters usa HDFS porque Spark precisa de um sistema de arquivos distribuído (DFS) existente para armazenar e acessar dados distribuídos.

## 4. Teste em Modo Local vs. Cluster Real

- No modo local (`local[*]`), todos os recursos estão na mesma máquina, e a latência de comunicação é mínima.
- Em um cluster real, o ganho de desempenho pode ser menor devido a:
  1. Latência de Rede: RDDs distribuídos requerem comunicação.
  2. I/O Distribuído: Acesso ao DFS, como HDFS.
  3. Overhead do Cluster Manager: Coordenação entre Driver e Executors.

## 5. Verdadeiro/Falso sobre RDDs

1. RDDs não são tolerantes a falhas. **Falso.** RDDs são *fault-tolerant*, reconstruídos automaticamente via *Lineage*.
2. Um RDD pode ser criado a partir de coleções existentes como uma lista Python. **Verdadeiro.** Ex.: `sc.parallelize([1,2,3])`.
3. Podemos criar RDDs lendo um arquivo do disco local. **Verdadeiro.**
4. RDDs não são particionados por padrão. **Falso.** Spark particiona automaticamente; o número de partições define o paralelismo máximo.

## 6. Output do Programa PySpark

```
rdd = sc.parallelize([1, 2, 3, -1, -2, -3])
rdd.filter(lambda x: x >= 0)\
    .map(lambda x: (1, x) if x % 2 == 0 else (2, x))\
    .reduceByKey(lambda x, y: x + y).collect()
```

- **Input RDD:** [1, 2, 3, -1, -2, -3]
- **filter(lambda x: x >= 0):** Remove negativos → [1,2,3]
- **map(...):** Cria pares (Chave, Valor)
  - 1 (par): (1,2)
  - 2 (ímpar): (2,1), (2,3)
- **reduceByKey(lambda x, y: x + y):** Soma os valores por chave
  - Chave 1 → (1,2)
  - Chave 2 → 1+3=4 → (2,4)

- **collect():** Retorna ao driver
- **Output:** [(1, 2), (2, 4)] (ordem pode variar)

## 7. Transformações Lazy (Preguiçosas)

- Operações Spark: **Transformações (lazy)** e **Ações (forçam execução)**.
- Exemplo:

1. `rdd = sc.parallelize(...)` → Lazy (salva como criar o RDD)
2. `print(rdd.collect())` → Não Lazy, ação
3. `rdd.filter(lambda x: x < 10)` → Lazy, transformação
4. `rdd.mapPartitions(lambda x: x)` → Lazy, transformação

## 8. Programa Spark para Classificação de Notas

```
rdd = sc.parallelize([('Lam', 45), ('Khiv', 64),
                    ('Alexi', 69), ('Kavan', 87),
                    ('Rebecca', 95)])
```

```
def classify_mark(record):
    name = record[0]
    mark = record[1]

    if mark < 50:
        classification = 'Fail'
    elif 50 <= mark <= 59:
        classification = 'Pass'
    elif 60 <= mark <= 69:
        classification = 'Merit'
    else: # 70 <= mark <= 100
        classification = 'Distinction'

    return (name, classification)
```

```
# Aplica a transformação map e força execução
result_rdd = rdd.map(classify_mark).collect()
# Output esperado:
# [('Lam', 'Fail'), ('Khiv', 'Merit'),
#  ('Alexi', 'Merit'), ('Kavan', 'Distinction'),
```



```
# ('Rebecca', 'Distinction')]
```

# Código da Prática 4

## Configuração Inicial do Spark

O código começa com a configuração do ambiente Spark para garantir a execução eficiente dos processos distribuídos.

```
1 from pyspark import SparkConf, SparkContext
2
3 conf = SparkConf().setMaster("local[*]").setAppName("Spark Lab")
4 sc = SparkContext(conf=conf)
```

### Notas de Estudo:

- `SparkConf()` e `SparkContext`: estabelecem a conexão inicial com o cluster (neste caso, `local[*]`, utilizando todos os núcleos disponíveis da máquina local).
- O objeto `sc` é o ponto de entrada para trabalhar com RDDs.

## Desenvolvimento do Algoritmo Word Count

A função `word_count` é desenvolvida sequencialmente para ilustrar a importância da transformação `flatMap`.

### Leitura e Erro Comum (`map`)

O primeiro passo lê o ficheiro e cria um RDD onde cada elemento corresponde a uma linha do texto.

```
1 def word_count(file_path):
2     lines = sc.textFile(file_path)
3
4     # Erro comum: map retorna um RDD de listas (nao achatado)
5     words = lines.map(lambda line: line.split(' '))
6
7     output = words.take(10)
8     return output
```

**Nota Crucial:** O uso de `map` resulta num RDD onde cada elemento é uma lista de palavras (por exemplo, `[['o', 'sol'], ['é', 'amarelo']]`), o que impede a contagem direta de palavras individuais.

### Uso Correto do `flatMap` (Transformação Essencial)

Para transformar o RDD de linhas num RDD de palavras individuais, utiliza-se `flatMap`.

```

1 def word_count(file_path):
2     lines = sc.textFile(file_path)
3
4     # Correcao: flatMap achata o RDD
5     words = lines.flatMap(lambda line: line.split(' '))
6
7     output = words.take(10)
8     return output

```

**Nota de Estudo:** O `flatMap` aplica a função de divisão e garante que o resultado seja devolvido como um RDD plano, sendo fundamental em operações textuais.

### Pipeline Completo de Contagem de Palavras (Aggregation Pattern)

O algoritmo final de *Word Count* segue o padrão MapReduce usando RDDs.

```

1 def word_count(file_path):
2     lines = sc.textFile(file_path)
3
4     words = lines.flatMap(lambda line: line.split(' '))
5
6     # 1. Filtrar strings vazias
7     words = words.filter(lambda word: word != '')
8
9     # 2. Mapear para pares (palavra, 1)
10    words = words.map(lambda word: (word, 1))
11
12    # 3. Reducao: soma dos valores por chave
13    words = words.reduceByKey(lambda a, b: a + b)
14
15    # 4. Ordenar por frequencia (decrecente)
16    words = words.sortBy(lambda x: x[1], ascending=False)
17
18    output = words.take(10)
19    return output

```

#### Notas de Estudo:

- `map(lambda word: (word, 1))` prepara os dados para agregação.
- `reduceByKey` executa a agregação de forma eficiente e distribuída.
- `sortBy` e `take(10)` produzem o resultado final.

## Pré-processamento Avançado

Para melhorar a precisão da contagem, o código introduz limpeza de caracteres e remoção de *stop words*.

### Função de Limpeza

```
1 import re
2
3 def remove_punctuation_symbols(text):
4     """Remove pontuacao e converte para minusculas"""
5     accepted_chars = re.compile(r'[^0-9a-zA-Z]+')
6     text = accepted_chars.sub(' ', text.lower()).strip()
7     return text
8
9 def word_count_cleaned(file_path):
10     lines = sc.textFile(file_path)
11     lines = lines.map(remove_punctuation_symbols)
12     # segue com flatMap, filtros e contagem
```

**Nota:** A limpeza é aplicada por linha (map) antes do flatMap, garantindo que a pontuação não seja incorporada nas palavras.

### Filtro de *Stop Words*

A remoção de palavras comuns é feita através de um filtro adicional após a divisão.

```
1 # Apos flatMap e remocao de strings vazias
2 words = words.filter(lambda word: word not in stop_words)
3
4 # Contagem final (map, reduceByKey, sortBy)
```

**Nota:** O filtro é aplicado apenas quando o RDD já contém palavras individuais e limpas.

## Análises Estatísticas com RDDs

### Histograma de Repetições

O objetivo é contar quantas palavras diferentes apresentam a mesma frequência.

```
1 def histogram_reps(file_path):
2     # word_counts: (palavra, contagem)
3
4     histogram = word_counts.map(lambda x: (x[1], 1))
```

```

5     histogram = histogram.reduceByKey(lambda a, b: a + b)
6
7     output = histogram.sortBy(lambda x: x[0], ascending=True).
        collect()
8     return output

```

**Nota:** Esta técnica inverte as chaves do RDD, permitindo calcular a frequência das frequências.

## Cálculo da Média do Comprimento das Palavras

O cálculo manual da média ilustra o padrão de agregação composta.

```

1 def average_length(file_path):
2     # word_lengths: RDD de inteiros
3
4     sum_and_count = word_lengths.map(lambda l: (l, 1)) \
5         .reduce(lambda a, b: (a[0] + b[0],
6                                a[1] + b[1]))
7
8     average = sum_and_count[0] / sum_and_count[1]
9     return average

```

**Nota de Estudo:** Para agregados compostos, utiliza-se uma tupla acumuladora. Alternativamente, pode-se usar diretamente `word_lengths.mean()`.

# Resumo do Capítulo 5: SparkSQL

## Introdução e Motivação: Porquê DataFrames?

O desenvolvimento do Spark SQL e dos DataFrames visou superar as limitações de otimização dos RDDs.

- **Limitações dos RDDs:** As transformações em RDDs são como "caixas pretas" (*black-boxes*) para o Spark, pois ele não conhece o tipo interno (T) nem a computação exata que está a ocorrer. Esta flexibilidade extrema limita as otimizações que o Spark pode aplicar.
- **Sobrecarga de Serialização:** A distribuição de dados e o armazenamento de RDDs implicam serialização Java ou Kryo, o que gera uma sobrecarga (*overhead*) de armazenamento, envio e Garbage Collector para cada objeto.
- **Ideia Chave do Spark SQL:** Impor estrutura aos dados para otimizar a execução. Isto permite ao Spark saber mais sobre os dados subjacentes.

## Abstrações Estruturadas: DataFrames e Datasets

DataFrames e Datasets são as APIs estruturadas no Spark SQL que permitem otimizar o processamento de Big Data.

### DataFrames ( $\geq$ Spark 1.3)

- São RDDs de linhas (*rows*) com colunas que podem ser acedidas pelos seus nomes.
- Semelhantes aos DataFrames do *Pandas* (Python).
- Evitam a serialização Java realizada pelos RDDs.
- Um DataFrame é uma coleção distribuída de elementos do tipo **Row**.
- Tal como os RDDs, os DataFrames são imutáveis.
- Requerem um *schema* para descrever os dados.

### Datasets ( $\geq$ Spark 1.6)

- São RDDs fortemente tipados (*strongly typed*).
- Combinam as vantagens dos RDDs (programação orientada a objetos, segurança de tipo em tempo de compilação) e dos DataFrames (otimização).
- Disponíveis apenas para Scala e Java.

## Fusão (Spark 2.0)

Desde novembro de 2016, um `DataFrame` é, na prática, um `Dataset de Rows` (`Dataset [Row]`).

## Arquitetura de Otimização

O poder das APIs estruturadas reside nos projetos internos do Spark que permitem a otimização:

1. **Catalyst Optimizer:** Otimiza o plano de execução.
2. **Tungsten Project:** Melhora a gestão da memória, permitindo uma gestão eficiente da memória *off-heap*.
3. **Performance Multilingue:** Graças ao Catalyst, o código escrito em Python, Scala ou Java é traduzido para o mesmo código Java subjacente, garantindo a mesma performance em todas as linguagens.

**Princípio de Otimização:** O Spark consegue otimizar o funcionamento interno porque trabalha por colunas e entende a intenção da operação (por exemplo, `col["x"] == 1`), ao contrário de uma função *lambda* opaca num RDD.

## Trabalhando com DataFrames

### Ponto de Entrada e Estrutura

- **SparkSession:** É o novo ponto de entrada unificado para a funcionalidade Spark (substituindo `SQLContext`, `SparkContext`, etc.).
- **Schema:** O esquema pode ser definido de três formas:
  1. Inferido automaticamente (ex.: de ficheiros JSON ou RDDs)
  2. Inferido de metadados (ex.: JDBC)
  3. Explicitamente definido usando `StructType`

## Operações (Transformações vs. Ações)

| Tipo           | Característica                                                | Exemplos Chave                                                                                                                                                                                                                                           |
|----------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transformações | Operações <i>lazy</i> (preguiçosas); criam um novo DataFrame. | <code>select()/selectExpr()</code> ,<br><code>filter()/where()</code> ,<br><code>orderBy()/sort()</code> ,<br><code>withColumn()</code> , <code>drop()</code> ,<br><code>distinct()</code> , <code>union()</code> ,<br><code>join(other, on, how)</code> |
| Ações          | Disparam a computação e devolvem o resultado ao Driver.       | <code>show()</code> , <code>count()</code> , <code>collect()</code> ,<br><code>first()</code> , <code>take(n)</code> , <code>toPandas()</code> ,<br><code>describe()</code>                                                                              |

- **Agregações:** Feitas através de `groupBy()` (que retorna um objeto `GroupedData`) seguido por funções como `count()`, `avg()`, `max()`, ou através da função `agg(*exprs)`.

## Funções Otimizadas

- Deve-se sempre tentar usar as mais de 100 funções nativas e otimizadas fornecidas pelo Spark SQL (ex.: para manipulação de strings ou tempo/data) para tirar o máximo partido do Tungsten e Catalyst.
- **UDFs (Funções Definidas pelo Utilizador):** Podem ser usadas para transformar DataFrames, mas não são tão eficientes quanto as funções nativas do Spark.

## Utilização de SQL

- Os DataFrames podem ser registados como vistas temporárias SQL:

```
df.createOrReplaceTempView("nome_da_view")
```

permitindo a execução de consultas SQL que beneficiam das mesmas otimizações.

## Filosofia e Advertências

- **DataFrames vs. RDDs (Filosofia):**
  - RDDs: Permitem-nos decidir **como** executar as operações.
  - DataFrames/Datasets: Permitem-nos definir **o que** queremos fazer, deixando o Spark decidir como otimizar a execução.
- **Caching:** O caching não é automático; deve ser usado `cache()` em DataFrames que serão reutilizados para armazenar o resultado em memória.



- **Interoperabilidade:** DataFrames são construídos sobre RDDs e são interoperáveis com eles.
- **Advertências Finais:** Nunca usar `collect()` ao lidar com grandes datasets, pois o resultado pode não caber na memória principal do Driver. Em vez disso, usar `take(n)`.
- **Importância:** DataFrames tornaram-se a API primária para Machine Learning no Spark.

## Ficha 5: SparkSQL

### Questão 1

- a. **Falsa.** RDDs não serão descontinuados e são interoperáveis com DataFrames. DataFrames tornaram-se a principal API para Machine Learning.
- b. **Falsa.** O Catalyst Optimizer garante o mesmo desempenho em todas as linguagens (Scala, Java, Python), traduzindo tudo para o mesmo código Java.
- c. **Falsa.** DataFrames são construídos sobre RDDs, que mantêm a linhagem para garantir a tolerância a falhas. RDDs são o mecanismo subjacente.
- d. **Verdadeira.** O Dataset API (como tipo forte) é exclusivo de Scala e Java, sendo o DataFrame (`Dataset[Row]`) a API Python.

### Questão 2

RDDs são caixas-pretas e suas transformações são funções lambda desconhecidas. O Spark SQL impõe um esquema (estrutura) aos dados, permitindo otimizações através dos projetos Catalyst e Tungsten.

### Questão 3

Não. O objeto `Row` é um array único que pode ser indexado por nome. É o DataFrame (a coleção) que é distribuído pelos *workers* do cluster.

### Questão 4

O esquema define os nomes e tipos de dados das colunas. Seu propósito é impor estrutura para que o Spark possa otimizar a execução, usando Tungsten e Catalyst, e evitar a serialização lenta do Java.

### Questão 5

- a. **Verdadeira.**
- b. **Verdadeira.** DataFrames podem ser criados a partir de coleções Python existentes no programa driver, como tuplas ou listas, ou RDDs.
- c. **Verdadeira.** DataFrames são construídos sobre RDDs, permitindo que o RDD subjacente seja acessado (e transformado) através da operação `.rdd`.
- d. **Falsa.** Inferir o esquema para grandes datasets pode ser muito lento, exigindo tempo de processamento. A definição manual (ou via metadados) pode ser mais rápida.

### Questão 6

Para visualizar o conteúdo, é necessário usar uma transformação como `select()` para selecionar a coluna e, em seguida, executar uma ação como `show()` ou `collect()`.

### Questão 7

- a. `df = rdd.toDF(['module', 'mark'])`: **Preguiçosa (Lazy)**. É uma transformação (conversão de RDD para DataFrame).
- b. `df.count()`: **Não preguiçosa (Ação)**. A ação `count()` força a execução de todas as transformações precedentes e retorna o resultado ao driver.
- c. `print(df.collect())`: **Não preguiçosa (Ação)**. O `collect()` é uma ação que executa o cálculo do DataFrame e retorna todos os dados ao *driver*.
- d. `df2 = df.groupBy('module').count()`: **Não preguiçosa (Ação)**. Embora `groupBy` seja uma transformação, o `count()` aplicado ao resultado é uma ação de agregação que força a execução.

### Questão 8

- a. **Falsa**. UDFs não são a escolha preferida, pois não são tão eficientes quanto as funções nativas do Spark que aproveitam as otimizações do Catalyst e Tungsten.
- b. **Verdadeira**. Operações de coluna retornam expressões, exigindo transformações. UDFs podem ser projetadas para serem usadas diretamente em transformações como `select()`.
- c. **Falsa**. DataFrames são imutáveis; não há transformações nem cláusulas SQL (como `DELETE`) que modifiquem permanentemente o conteúdo de um DataFrame existente no Spark.

# Resumo do Capítulo 6: ML com Spark

## O Desafio do Machine Learning em Big Data

O ML em Big Data exige ferramentas que vão além dos frameworks tradicionais.

## Recapitulação de Frameworks de Big Data

Frameworks como o Spark fornecem ferramentas essenciais para:

- Processamento iterativo, interativo e de streaming.
- Baseiam-se no paradigma de programação MapReduce (e suas extensões).
- Utilizam Sistemas de Arquivos Distribuídos, aplicando o princípio da localidade de dados.
- Garantem tolerância a falhas (fault-tolerance), crucial para execuções longas.

## Desafios e a Ciência do ML Distribuído

A extração de conhecimento em Big Data é complexa devido:

- **Escala Crescente:** Tanto ao nível das instâncias quanto ao nível das features (características).
- **Velocidade:** A rapidez com que os dados chegam (velocity).
- **Variedade e Complexidade** do problema.

A "ciência" de fazer ML em Big Data reside na forma como a computação é distribuída para minimizar o movimento de dados e maximizar o aprendizado em um tempo razoável. Isso implica adaptar métodos existentes e redesenhar etapas que são intratáveis com grandes volumes de dados.

## Bibliotecas de Machine Learning

Bibliotecas tradicionais (e.g., scikit-learn) são projetadas para processamento sequencial, não escalam horizontalmente e não oferecem tolerância a falhas. Em contraste, o Spark oferece a MLlib.

## Spark MLlib

A MLlib explora a API DataFrames do Spark e oferece:

- Algoritmos tradicionais para classificação, regressão, agrupamento (clustering) e filtragem colaborativa.
- Métodos de transformação de features (e.g., extração, seleção, redução de dimensionalidade).
- Ferramentas de Pipelines para construir e avaliar fluxos de trabalho de ML.

Outras bibliotecas relevantes para ML em Big Data incluem FlinkML e DaskML.

## Conceitos Básicos da MLlib: Pipelines

Um Pipeline é uma estrutura de programação que automatiza partes do ciclo de vida do ML, sendo uma sequência concatenada de Transformers e Estimators.

### Transformer

- É um algoritmo que transforma um DataFrame em um novo DataFrame, por exemplo, adicionando uma ou mais colunas.
- Implementa o método `transform()`.
- Exemplos incluem: métodos de pré-processamento sem parâmetros (e.g., one-hot encoding), métodos de pré-processamento com parâmetros previamente ajustados (e.g., max-min scaler), ou um modelo de ML já aprendido usado para fazer previsões.
- Não precisa ajustar parâmetros.

### Estimator

- Pode ser qualquer algoritmo de ML, ou um algoritmo que ajusta alguns parâmetros.
- Recebe um DataFrame como entrada e ajusta os parâmetros.
- Implementa o método `fit()`.
- **Ajusta parâmetros e retorna um Transformer.**
- Exemplos incluem: um método de ML (e.g., Regressão Linear ajustando seus parâmetros) ou uma técnica de pré-processamento que requer o ajuste de parâmetros (e.g., um max-min scaler).

## O Pipeline como Estrutura

- Um Pipeline é, ele próprio, um Estimator (possui um método `fit()`).
- O resultado de ajustar (*fitting*) um Pipeline é um Transformer, que pode ser usado para realizar previsões.

## Ficha 6: ML com Spark

### Questão 1: Verdadeiro ou Falso

- a. **Falsa.** O pré-processamento envolve transformações de dados (e.g., feature selection ou one-hot encoding), não a distribuição uniforme dos dados brutos em um cluster.
- b. **Verdadeira.** Estratégias de avaliação (como CrossValidator ou TrainValidationSplit) são cruciais para o ajuste (tuning) dos hiperparâmetros de um modelo.
- c. **Falsa.** Um Pipeline é uma estrutura de programação que concatena Transformers e Estimators, automatizando o fluxo de trabalho do ML, mas não seleciona o algoritmo ideal.
- d. **Verdadeira.** O overfitting ocorre quando o modelo memoriza o ruído dos dados de treino, resultando em um desempenho insatisfatório ao ser aplicado a dados não vistos.

### Questão 2: Aspectos Chave do ML em Big Data

Deve-se considerar a necessidade de longas execuções (exigindo tolerância a falhas), a necessidade de pré-transformação de dados, e que muitos algoritmos requerem múltiplas iterações sobre os dados.

### Questão 3: Preferência Spark vs. Hadoop para ML

Spark armazena dados em memória, o que é mais rápido do que o I/O intensivo em disco do Hadoop MapReduce. Isso é crucial para algoritmos de ML iterativos que revisitam os dados várias vezes.

### Questão 4: Precisão do Modelo em Cluster Real

Sim, espera-se que a precisão permaneça a mesma. O MLlib usa abordagens globais que treinam um único modelo com todos os dados, sendo o resultado final independente do número de nós ou partições.

# Código da Prática 6

## Configuração e Preparação de Dados

### Inicialização e Leitura

O código começa por inicializar a sessão Spark, que é o ponto de entrada para o uso de DataFrames, e por carregar o conjunto de dados.

```
1 from pyspark.sql import SparkSession
2
3 spark = SparkSession.builder \
4     .master("local[*]") \
5     .appName("MLlib Lab") \
6     .getOrCreate()
7
8 sc = spark.sparkContext
9
10 # Leitura do CSV e cache
11 df = spark.read.csv(
12     "content/energydata_complete.csv",
13     header=True
14 ).cache()
15
16 df.columns
17 df.show(5)
```

### Notas de Estudo:

- SparkSession é a API moderna para interagir com o Spark.
- O uso de `cache()` é crucial em algoritmos de *Machine Learning*, que são iterativos e reutilizam os dados múltiplas vezes.

### Engenharia de Features e Limpeza

As features temporais são extraídas da coluna `date`, e os tipos de dados são convertidos para o formato numérico esperado pelo MLlib.

```
1 import pyspark.sql.functions as sql_f
2 from pyspark.sql.functions import col
3 from pyspark.sql.types import FloatType
4
5 # Extração de features temporais
6 df = df.withColumn("dayofyear", sql_f.dayofyear("date"))
7 df = df.withColumn("hour", sql_f.hour("date"))
```



```

8 df = df.withColumn("month", sql_f.month("date"))
9
10 df = df.drop("date")
11
12 # Conversao de colunas para Float
13 for c in df.columns:
14     if c not in ["dayofyear", "hour", "month"]:
15         df = df.withColumn(c, col(c).cast(FloatType()))
16
17 df.printSchema()

```

### Notas de Estudo:

- Por defeito, o Spark lê colunas numéricas de ficheiros CSV como `string`.
- A conversão explícita para `FloatType` é necessária para utilização em modelos de MLlib.

### Divisão de Dados e Análise Exploratória

O conjunto de dados é dividido em treino e teste, sendo realizada uma breve análise exploratória da variável alvo (`Appliances`).

```

1 seed = 123456
2
3 # Divisao dos dados
4 df_train, df_test = df.randomSplit([0.7, 0.3], seed=seed)
5
6 # Histograma de consumo por mes (treino)
7 hist_elect = (
8     df_train.groupBy("month")
9     .sum("Appliances")
10    .sort("month")
11    .collect()
12 )
13
14 examples_per_month = (
15     df_train.groupBy("month")
16     .count()
17     .sort("month")
18     .show()
19 )
20
21 # Remocao da coluna 'month' antes do treino
22 df = df.drop("month")

```

```
23 df_train = df_train.drop("month")
24 df_test = df_test.drop("month")
```

**Nota:** A coluna `month` é removida antes do treino, possivelmente por ser considerada redundante após a análise exploratória ou para simplificar a modelação inicial.

## Construção e Treinamento do Modelo (Pipeline 1)

### Definição do Pipeline MLlib

É construído um pipeline que agrega as features, define o modelo de regressão e inclui validação cruzada para ajuste de hiperparâmetros.

```
1 from pyspark.ml.feature import VectorAssembler
2 from pyspark.ml.regression import GBTRegressor
3 from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
4 from pyspark.ml.evaluation import RegressionEvaluator
5 from pyspark.ml import Pipeline
6
7 # Agregacao das features
8 feature_columns = [c for c in df.columns if c != "Appliances"]
9
10 vectorAssembler = VectorAssembler(
11     inputCols=feature_columns,
12     outputCol="features"
13 )
14
15 # Modelo GBT
16 gbt = GBTRegressor(labelCol="Appliances")
17
18 # Grid de hiperparametros
19 paramGrid = (
20     ParamGridBuilder()
21     .addGrid(gbt.maxDepth, [5, 10])
22     .addGrid(gbt.maxIter, [20, 50])
23     .build()
24 )
25
26 # Avaliador (RMSE)
27 evaluator = RegressionEvaluator(
28     labelCol="Appliances",
29     predictionCol="prediction",
30     metricName="rmse"
31 )
```

```

32
33 # Cross-validation
34 cv = CrossValidator(
35     estimator=gbt,
36     estimatorParamMaps=paramGrid,
37     evaluator=evaluator
38 )
39
40 # Pipeline
41 pipeline = Pipeline(stages=[vectorAssembler, cv])

```

### Notas de Estudo:

- O VectorAssembler é obrigatório para converter múltiplas colunas num único vetor de features.
- O CrossValidator ajusta múltiplos modelos e seleciona o melhor com base no RMSE.

## Treinamento e Avaliação Inicial

```

1 # Treinamento
2 pipeline_model = pipeline.fit(df_train)
3 pipeline_model.write().overwrite().save("pipeline_model")
4
5 # Avaliacao no conjunto de teste
6 predictions = pipeline_model.transform(df_test)
7 predictions.select("Appliances", "prediction").show(5)
8
9 rmse = evaluator.evaluate(predictions)
10 rmse

```

### Notas de Estudo:

- O método fit executa todas as etapas do pipeline.
- O modelo treinado é persistido para reutilização futura.

## Análise de Importância de Features e Retreino (Pipeline 2)

A importância das features do modelo GBT é utilizada para realizar seleção de variáveis.

```

1 # Aceder ao CrossValidatorModel
2 cv_model = pipeline_model.stages[1]
3

```

```

4 # Importancia das features
5 importances = cv_model.bestModel.featureImportances
6
7 # Remocao de features pouco relevantes
8 to_remove = [
9     feature_columns[i]
10     for i, importance in enumerate(importances)
11     if importance < 0.05
12 ]
13
14 print(to_remove)
15
16 # Novo VectorAssembler
17 vectorAssembler2 = VectorAssembler(
18     inputCols=[c for c in feature_columns if c not in to_remove],
19     outputCol="features"
20 )
21
22 # Segundo pipeline
23 pipeline2 = Pipeline(stages=[vectorAssembler2, cv])
24 pipeline_model2 = pipeline2.fit(df_train)
25
26 # Avaliacao final
27 predictions = pipeline_model2.transform(df_test)
28 rmse = evaluator.evaluate(predictions)
29 rmse

```

### Notas de Estudo:

- Modelos baseados em árvores fornecem métricas de importância das features.
- A seleção de features pode melhorar desempenho e reduzir complexidade.
- O segundo pipeline demonstra um ciclo completo de refinamento do modelo.

# Resumo do Capítulo 7: ML em Big Data: Global vs. Local e Escalabilidade

## Abordagens de Machine Learning em Big Data

Existem duas abordagens distintas para projetar soluções de Machine Learning (ML) em ambientes de Big Data.

### Modelos Locais

Os modelos locais exploram o fato de que os dados estão distribuídos em partições, aplicando o algoritmo de ML original localmente em cada fragmento de dados.

- **Mecanismo:** É aplicada a estratégia de *dividir e conquistar* (Divide-and-conquer), onde um modelo é criado para cada partição, usando apenas os dados dessa partição.
- **Combinação:** Os modelos individuais são combinados (usualmente através de um *ensemble*, como votação majoritária) para prever a classe de um novo exemplo.
- **Vantagens:** Geralmente mais rápidos na fase de treinamento e escalam bem com o aumento do número de partições. Qualquer modelo existente pode ser facilmente aplicado, sendo necessário apenas projetar a fase de agregação.
- **Desvantagens:** Lentidão na fase de teste (muitos modelos precisam ser executados). A precisão pode deteriorar se o número de partições for muito alto, pois pode não haver informação suficiente em cada partição para um aprendizado eficaz. Não utilizam todos os dados em conjunto para construir um único modelo coeso.

### Modelos Globais

Os modelos globais visam replicar o comportamento da versão sequencial do algoritmo, olhando para os dados como um todo e aprendendo um único modelo abrangente.

- **Mecanismo:** Exige o conhecimento profundo do algoritmo para paralelizar suas operações. O objetivo é construir um único modelo que use todos os dados disponíveis simultaneamente.
- **Vantagens:** Maior precisão esperada, pois todos os exemplos são usados para aprender um único modelo. O modelo final é independente do número de partições, e a fase de teste é mais rápida (apenas um modelo para executar).
- **Desvantagens:** O design e a implementação são mais complexos, sendo necessário levar em conta a natureza distribuída (custos de comunicação/cálculo).

## Árvores de Decisão (DT) como Exemplo

As Árvores de Decisão são técnicas de ML conhecidas por serem interpretáveis, capazes de lidar com dados numéricos e categóricos, e são frequentemente usadas em métodos de *ensemble*.

### Implementação Local de DT

A abordagem local para DTs em Big Data resulta em um *ensemble* (conjunto de modelos) em vez de uma única árvore.

- **Fase de Construção:** Utiliza o Spark RDD API, especificamente `mapPartitions()`, para criar um modelo de árvore em cada partição de dados.
- **Fase de Teste:** Cada modelo faz uma previsão, e um voto majoritário entre as previsões é calculado para obter o resultado final.

### Implementação Global de DT (MLlib)

A implementação Global do Spark MLlib lida com dois problemas chave da paralelização de DTs:

- **Recursão:** A construção recursiva da árvore pode gerar um número excessivo de jobs. A solução é construir a árvore **nível por nível**. Todos os nós de um nível são aprendidos em uma única passagem pelo conjunto de dados.
- **Atributos Contínuos:** Atributos numéricos podem ter um número muito grande de pontos de corte candidatos. A solução é usar uma **estratégia de binning** (agrupamento em caixas) para limitar o número de divisões possíveis.

O modelo global é mantido no driver e é atualizado e transmitido (*broadcasted*) aos executors em cada iteração.

## Medidas de Escalabilidade em Sistemas Distribuídos

A escalabilidade de uma solução de Big Data deve ser testada empiricamente para garantir que ela possa lidar com conjuntos de dados arbitrariamente grandes, considerando fatores como sobrecarga de rede, caching e o número de jobs do Spark.

### Métricas Clássicas

- **Speed-up (Aceleração):** Mede quão mais rápido o mesmo volume de dados pode ser processado usando N núcleos em vez de 1 núcleo (tamanho dos dados fixo, variação de núcleos). Uma aceleração linear significa que o tempo de execução é inversamente proporcional ao número de núcleos.

- **Size-up (Aumento de Tamanho):** Mede quanto tempo leva para processar um conjunto de dados  $N$  vezes maior, mantendo o número de núcleos fixo (tamanho dos dados variável, núcleos fixos). Uma linearidade aqui é rara devido à complexidade inerente de muitos algoritmos de ML.
- **Scale-up (Escala Proporcional):** Mede a capacidade de um sistema rodar um conjunto de dados  $N$  vezes maior com um sistema  $N$  vezes maior (aumento proporcional de dados e núcleos). O valor perfeito é 1, indicando que o tempo de processamento permanece constante.

## Considerações na Medição de Tempo de Execução

Para medir corretamente o tempo de execução, é crucial:

- Medir apenas o tempo de execução dentro do programa (desconsiderando o tempo de inicialização da `SparkSession`).
- Forçar ações nas operações lazy do Spark para que o tempo de cálculo seja contabilizado.
- Usar uma linha de base significativa para o cálculo das métricas, evitando baselines que contenham uma porcentagem desproporcionalmente alta de sobrecarga do Spark.

## Análise de Escalabilidade em DT Locais

A complexidade das Árvores de Decisão é tipicamente  $O(N \log N)$  (onde  $N$  é o número de instâncias), o que sugere um comportamento não linear em relação ao tamanho dos dados.

## Resultados Esperados vs. Encontrados (DT Local)

- **Speed-up:** Esperava-se um resultado super-linear, pois o aumento de núcleos reduziria o tamanho dos dados em cada partição, tornando cada modelo individual mais rápido. Empiricamente, o resultado foi melhor que linear (desconsiderando a sobrecarga do Spark), mas não super-linear, devido a fatores como a sobrecarga crescente do Spark.
- **Size-up:** Esperava-se um resultado pior que linear (o tempo cresceria mais rápido que o tamanho dos dados) devido à complexidade  $O(N \log N)$ , o que foi confirmado empiricamente.
- **Scale-up:** Esperava-se um comportamento linear, pois o aumento de núcleos deveria compensar o aumento de dados. Os resultados empíricos não confirmaram essa linearidade, sugerindo influência da arquitetura do sistema operacional ou da incapacidade do Spark de explorar totalmente o potencial computacional.

Em análises com limitações de hardware, o modelo Local de Árvores de Decisão demonstrou ser significativamente melhor em escalabilidade do que sua contraparte Global (MLlib).



## Ficha 7: ML em Big Data: Global vs. Local e Escalabilidade

### Questão 1: Verdadeiro ou Falso (Abordagens Locais)

- a. **Verdadeira.** A precisão e a fase de teste são afetadas pelo número de partições, pois isso determina a quantidade de dados usada para treinar cada modelo local.
- b. **Verdadeira.** Se houver muitas partições (modelos), a fase de teste é lenta, pois é necessário executar todos os modelos individuais para calcular a votação majoritária.
- c. **Falsa.** A abordagem local gera um conjunto (*ensemble*) de modelos, sendo um modelo treinado independentemente em cada partição de dados.
- d. **Falsa.** Se o número de partições for pequeno, a abordagem local pode beneficiar-se do efeito de conjunto, resultando em melhorias no desempenho.

### Questão 2: Solução Local para Qualquer Algoritmo

Sim. A solução local utiliza a estratégia de dividir e conquistar, aplicando o algoritmo original localmente em cada partição. Apenas a fase de agregação (combinação dos resultados) deve ser adaptada para cada algoritmo existente.

### Questão 3: Verdadeiro ou Falso (Abordagens Globais)

- a. **Verdadeira.** Os modelos globais visam replicar o comportamento sequencial do algoritmo original, aprendendo um único modelo usando todo o conjunto de dados.
- b. **Falsa.** A fase de teste é mais rápida em modelos globais porque apenas um modelo precisa ser executado, ao contrário da execução de múltiplos modelos locais.
- c. **Verdadeira.** Como o modelo é aprendido usando todos os dados de forma coesa, o desempenho de precisão é independente do número de partições.
- d. **Verdadeira.** Eles replicam o comportamento sequencial, mas muitas vezes possuem recursos limitados, como a necessidade de *binning* e a falta de *pruning* nas Árvores de Decisão.

### Questão 4: Diferença MLib DT vs. scikit-learn

A implementação do MLib usa *binning* para atributos contínuos e constrói a árvore nível por nível. Estas adaptações, necessárias para Big Data, podem levar a resultados diferentes da versão sequencial que usa todos os dados continuamente.

### Questão 5: Abordagem com Melhores Resultados

A abordagem Global deve obter melhores resultados, pois utiliza todos os exemplos para aprender um modelo coeso. No entanto, mais dados podem não ser úteis se o modelo não for atualizado ou adaptado, como ocorreu com o Google Flu Trends.

### Questão 6: DT Iterativa vs. Recursiva

A recursão gera um número excessivo de tarefas (*jobs*) no ambiente distribuído. A implementação do Spark utiliza uma abordagem iterativa, construindo a árvore nível por nível numa única passagem pelos dados para otimizar o processo.

### Questão 7: Manutenção da Precisão no Cluster

Sim, espera-se que a precisão permaneça a mesma. A regressão logística do MLlib é um modelo global, e sua precisão é independente do número de partições ou nós de computação utilizados.

### Questão 8: Treinamento e Teste de LR Local

O treino usaria `mapPartitions` para criar um modelo de regressão logística em cada partição de dados. O teste combinaria as previsões desses modelos (e.g., calculando a média das previsões ou votação majoritária) para obter o resultado final.

### Questão 9: Local Approach em Dados Desbalanceados

Seria arriscado. Se houver muitas partições, as 1.000 instâncias positivas podem ser diluídas ou ausentes em certas partições. Isto não forneceria informação suficiente para o modelo local aprender eficazmente, resultando em perda de precisão.

### Questão 10: Erro de Implementação do Modelo Local

O erro ocorre porque a criação de `SparkSession` e `createDataFrame` são operações destinadas ao driver. Além disso, o `Estimator (lr.fit)` não deve ser definido dentro da função `mapPartitions` executada nos workers.

### Questão 11: Verdadeiro ou Falso (Escalabilidade)

- a. **Verdadeira.** A complexidade, como  $O(N \log N)$  para Árvores de Decisão, é crucial para prever se a escalabilidade será linear ou não-linear em relação ao tamanho dos dados.
- b. **Falsa.** Em modo pseudo-distribuído (máquina única), a avaliação pode ser otimista, pois os custos de rede, que afetam a escalabilidade, não são considerados.

- c. **Falsa.** O conjunto de dados deve ser suficientemente grande para que a sobrecarga do Spark não distorça os resultados, especialmente ao definir a linha de base.
- d. **Falsa.** Se a versão sequencial for impossível devido ao tamanho dos dados, pode-se usar uma linha de base com um número maior de núcleos (e.g., 8) ou um subconjunto dos dados.

### **Questão 12: Principais Cuidados na Medição de Escalabilidade**

Os fatores incluem sobrecarga da rede, uso inadequado de *cache*, influência de outros processos do sistema operativo, e o alto custo da inicialização da SparkSession, o que pode enviesar o tempo de execução.

### **Questão 13: Cálculo e Discussão da Escalabilidade**

O Speed-up é sub-linear (e.g.,  $S_8 = 5.0$  para 8 núcleos), embora fosse esperado ser super-linear. O Scale-up (proporcional) degrada com o aumento, passando de 0.78 (2x) para 0.46 (8x). Isto sugere influência da sobrecarga do sistema.

# Código da Prática 7

## Configuração e Preparação de Dados

### Inicialização e Definição de *Schema*

O código inicializa a sessão Spark e define um *schema* explícito para garantir a correta ingestão dos dados de notas (`marks.csv`) como números de ponto flutuante, o que é crucial para o treinamento do modelo.

```
1 from pyspark.sql import SparkSession
2 import pyspark.sql.functions as sql_f
3 from pyspark.sql.types import *
4
5 spark = SparkSession.builder
6 .master("local[*]")
7 .appName("MLbig")
8 .getOrCreate()
9
10 sc = spark.sparkContext
11
12 schema = StructType([
13 StructField("Ex01", FloatType(), True),
14 # ... outras colunas de notas
15 StructField("Exam", FloatType(), True),
16 StructField("Total", FloatType(), True)
17 ])
18
19 df = spark.read.format('csv')
20 .option("header", 'true')
21 .schema(schema)
22 .load("data/marks.csv")
```

### Notas de Estudo:

- A definição explícita de `schema` com `FloatType()` evita que o Spark infira incorretamente os tipos de dados de um CSV, que frequentemente são lidos como *strings*.

### Engenharia de *Features* e Limpeza

A coluna alvo (*Classification*) é criada a partir da coluna `Total`. Em seguida, várias colunas intermédias são removidas, mantendo apenas as colunas de *features* agregadas e a nova variável alvo.

```
1 df = df.withColumn(
```

```

2 'Classification',
3 sql_f.when(df.Total < 70, 'not-first').otherwise('first')
4 )
5
6 df = df.drop("Question 1")
7 .drop("Question 2")
8 .drop("Question 3")
9 .drop("Question 4")
10 .drop("Exam")
11 .drop("Total")
12
13 df.show(2)

```

**Nota:** A classificação é binária: 'first' se a nota total for maior ou igual a 70 e 'not-first' caso contrário.

## Abordagem Local: Treinamento Distribuído com RDDs

Esta secção demonstra como treinar múltiplos modelos sequenciais (`DecisionTreeClassifier` do `scikit-learn`) em diferentes partições de dados e agregá-los para previsão.

### Divisão, Repartitionamento e Treinamento

O conjunto de treino é reparticionado para criar fragmentos de dados que serão processados de forma isolada nos *workers*.

```

1 from sklearn.tree import DecisionTreeClassifier
2 import pandas as pd
3
4 train_df, test_df = df.randomSplit([0.7, 0.3], seed=654321)
5
6 # O numero de particoes (3) define quantos modelos locais serao
   treinados
7
8 train_rdd = train_df.rdd.repartition(3)
9 test_rdd = test_df.rdd.repartition(2)
10
11 column_names = df.columns
12
13 def build_model(partition_data_it):
14 # O worker transforma o iterador RDD num DataFrame Pandas
15 partition_data_df = pd.DataFrame(partition_data_it, columns=
   column_names)
16

```

```

17 '''
18 clf = DecisionTreeClassifier(random_state=0, max_depth=2)
19
20 # Selecao local de features (X) e rotulos (y)
21 X_train = partition_data_df.iloc[:, :-1]
22 y_train = partition_data_df["Classification"]
23
24 model = clf.fit(X_train.values, y_train.values)
25 return [model]
26 '''
27
28 # Treino dos modelos locais
29
30 models = train_rdd.mapPartitions(build_model).collect()
31 len(models)

```

### Notas Cruciais:

- `mapPartitions` permite executar código não-Spark (como `scikit-learn`) em cada nó de forma isolada.
- `collect()` é utilizado para trazer os modelos treinados para o programa *driver*.

### Previsão e Agregação (Votação)

Na fase de teste, cada instância é avaliada por todos os modelos locais e o resultado final é obtido por voto maioritário (*ensemble*).

```

1 def predict(instance):
2     return [m.predict([instance[: -1]])[0] for m in models]
3
4 def agg_predictions(preds):
5     votes = {"first": 0, "not-first": 0}
6     for p in preds:
7         votes[p] += 1
8     return max(votes, key=votes.get)
9
10 # Aplicacao das previsoes no RDD de teste
11
12 test_rdd.map(predict).map(agg_predictions).take(5)

```

### Avaliação da Precisão da Abordagem Local

As previsões são reformuladas para que o avaliador do MLlib possa ser utilizado, exigindo a conversão de rótulos categóricos para valores numéricos.

```

1 from pyspark.sql import Row
2 from pyspark.ml.evaluation import MulticlassClassificationEvaluator
3
4 def transform(instance):
5     return Row(
6         **instance.asDict(),
7         raw_prediction=agg_predictions(predict(instance))
8     )
9
10 prediction = test_rdd.map(transform).toDF()
11
12 prediction_num = prediction.select(
13     (prediction["Classification"] == "first").cast("double").alias("
14         label"),
15     (prediction["raw_prediction"] == "first").cast("double").alias("
16         pred")
17 )
18
19 acc_evaluator = MulticlassClassificationEvaluator(
20     metricName="accuracy",
21     labelCol="label",
22     predictionCol="pred"
23 )
24
25 acc_evaluator.evaluate(prediction_num)

```

## Abordagem Global: Spark MLlib (Comparação)

Esta secção apresenta a implementação de um modelo global utilizando o `DecisionTreeClassifier` do Spark MLlib, para comparação com a abordagem local.

```

1 from pyspark.ml.feature import VectorAssembler, StringIndexer
2 from pyspark.ml.classification import DecisionTreeClassifier
3 from pyspark.ml import Pipeline
4
5 feature_cols = train_df.columns
6 feature_cols.remove('Classification')
7
8 assembler = VectorAssembler(inputCols=feature_cols, outputCol="
9     features")
10 labelIndexer = StringIndexer(
11     inputCol='Classification',

```

```

11 outputCol='ClassificationIndex'
12 )
13
14 dt = DecisionTreeClassifier(
15 labelCol='ClassificationIndex',
16 featuresCol='features'
17 )
18
19 pipeline = Pipeline(stages=[assembler, labelIndexer, dt])
20 pipeline_model = pipeline.fit(train_df)
21
22 prediction = pipeline_model.transform(test_df)
23
24 acc_evaluator = MulticlassClassificationEvaluator(
25 metricName="accuracy",
26 labelCol="ClassificationIndex",
27 predictionCol="prediction"
28 )
29
30 acc_evaluator.evaluate(prediction)

```

### Notas de Estudo:

- Esta abordagem treina um único modelo global coeso.
- O StringIndexer é necessário porque o MLlib exige rótulos numéricos.

## Análise de Escalabilidade: Cálculo de Métricas

O código estabelece o enquadramento para medir a escalabilidade da abordagem local, através da execução de um script externo e do cálculo de métricas clássicas.

### Definição das Métricas

```

1 def speed_up(results, runtime_col):
2     return results[runtime_col].iloc[0] / results[runtime_col]
3
4 def size_up(results, runtime_col):
5     return results[runtime_col] / results[runtime_col].iloc[0]
6
7 def scale_up(results, runtime_col):
8     return results[runtime_col].iloc[0] / results[runtime_col]

```



## Execução dos Experimentos de Speed-up

O *Speed-up* mede o ganho de desempenho ao processar o mesmo volume de dados com diferentes números de núcleos.

```
1
2 # Exemplo conceptual
3
4 for cores in num_cores:
5 !python code/scalability-localDT.py {cores} 100 speedup.csv
```

**Nota:** Faz-se uma distinção entre o tempo de execução total (*runtime*) e o tempo sem a sobrecarga do Spark (*runtime\_no\_overhead*), permitindo uma análise mais rigorosa da eficiência da paralelização.

# Resumo do Capítulo 8:

## Recapitulação de Abordagens de Big Data

O design de soluções de ML em Big Data segue duas estratégias principais:

- **Abordagem Global:** Visa um único modelo que utiliza o conjunto de dados completo. Supostamente mais preciso, mas requer um design complexo, pois é necessário saber como paralelizar o método original.
- **Abordagem Local (Divide-and-Conquer):** Cria um modelo para cada partição de dados e depois agrega os resultados. O resultado final depende do número de partições.

## Estudo de Caso I: K-means (Clustering)

K-means é um algoritmo de Aprendizado Não Supervisionado usado para descobrir a estrutura natural dos dados (agrupamento). É um algoritmo **iterativo** com passos bem definidos:

1. Inicializar K centros de cluster.
2. Repetir até a convergência:
  - Atribuir cada ponto de dados ao centro mais próximo.
  - Recalcular cada centro de cluster como a média dos pontos atribuídos.

## Implementação Global em MapReduce (Mahout)

A solução Mahout para K-means é um exemplo de algoritmo **iterativo MapReduce**.

- **Map:** Recebe os K centroids e um bloco de dados; atribui cada ponto ao centroid mais próximo; emite pares <centroid, ponto>.
- **Reduce:** Recebe todos os pontos para um centroid específico; recalcula o novo centroid (média).
- **Desafio:** Hadoop/MapReduce não são a plataforma ideal para algoritmos iterativos devido ao alto custo de startup de jobs e ao shuffle repetitivo de dados para o HDFS.

## Implementação Global em Spark

O Spark é superior para K-means devido à sua capacidade de computação em memória (RDDs) e tolerância a falhas.

- **Inicialização:** Os centros de cluster são amostrados (`takeSample`) a partir dos dados.
- **Atribuição (Map):** É utilizada uma operação `map` para calcular o ponto mais próximo de cada exemplo e gerar pares `<centroid, ponto>` (onde a chave é o índice do centroid).
- **Recálculo (Reduce/Agregação):** É utilizado `groupByKey()` para agrupar todos os pontos pertencentes a um centroid, seguido por `mapValues` para calcular a nova média, resultando nos novos centros.

## Estudo de Caso II: Regressão Linear

A Regressão Linear busca encontrar os parâmetros ( $\theta$ ) que minimizam a função de custo (tipicamente o Erro Quadrático Médio, MSE), assumindo que a variável alvo é uma combinação linear das características de entrada.

### Solução de Forma Fechada (*Closed-Form Solution*)

- A função de custo MSE é convexa, permitindo uma solução direta e exata.
- O problema de *Big Data* reside na computação de  $X^T X$  (onde  $X$  é a matriz de dados) e na sua inversão.
- **Cálculo de  $X^T X$  Distribuído:** Em vez de uma multiplicação matricial direta, utiliza-se a **soma dos produtos externos** (*sum of outer products*), onde o produto é calculado somando o produto externo de cada exemplo (linha) por si mesmo. Este método é eficiente para paralelização.
- **Inversão de Matriz:** A inversão de  $(X^T X)^{-1}$  (matriz de tamanho  $n \times n$ , sendo  $n$  o número de *features*) é calculada localmente no *driver*, assumindo que  $n$  é pequeno.
- **Limitação:** Se o número de *features* ( $n$ ) for muito grande, a inversão torna-se intratável (complexidade cúbica,  $O(n^3)$ ), e a solução de forma fechada falha.

### Descida de Gradiente (*Gradient Descent*)

É uma solução alternativa (aproximada) usada quando a solução de forma fechada é inviável devido a um grande número de *features*.

- **Princípio:** Inicia com valores aleatórios de  $\theta$  e atualiza-os iterativamente na direção que reduz a função de custo (direção oposta ao gradiente), usando um hiperparâmetro chamado taxa de aprendizado.

- **Paralelização:** O gradiente total é a média dos gradientes de cada exemplo, o que facilita a paralelização.
- **Processo Iterativo Distribuído:** Os parâmetros ( $\theta$ ) são transmitidos (*broadcasted*) aos *workers* em cada iteração. Os *workers* calculam os gradientes, que são agregados (média), e os parâmetros são recolhidos e atualizados pelo *driver*.
- **Otimização:** O **caching** dos dados é essencial, pois estes são reutilizados em cada iteração.

## Ficha 8:

### Questão 1: Princípios para Solução Global

- Deve-se replicar o comportamento sequencial do algoritmo usando todo o conjunto de dados.
- É crucial conhecer a complexidade e os passos do algoritmo para identificar gargalos e paralelizar as operações.
- A estratégia deve visar a escalabilidade linear, executando o máximo de operações em paralelo e fazendo *caching* de dados reutilizados.

### Questão 2: Dificuldades na Paralelização do K-means

O K-means é um algoritmo iterativo MapReduce. Em plataformas como o Hadoop, isso resulta em altos custos de inicialização de jobs, serialização e *shuffle* repetitivos, tornando-o ineficiente e lento.

### Questão 3: Abordagem Local para K-means

Sim, seria possível usar a estratégia *Divide-and-Conquer* (`mapPartitions`). Isso criaria um ensemble de modelos K-means locais. Espera-se perda de precisão se o número de partições for muito grande, devido à insuficiência de dados em cada partição.

### Questão 4: K-medoids (PAM) e Escalabilidade Global

O K-means é relativamente simples de paralelizar globalmente (calcula médias). O K-medoids, que usa pontos de dados existentes como representantes, tem etapas de atualização mais complexas que complicariam a paralelização e a escalabilidade global, exigindo um design mais complexo.